The Journal of Open Source Software

Aetherion: A structure-preserving, differentiable C++23 library for six-degree-of-freedom flight dynamics

Onur Tuncer [1¶]

1 Department of Aerospace Engineering, Istanbul Technical University, Türkiye ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Aetherion is a C++23 library for simulating the six-degree-of-freedom (6-DOF) motion of aircraft and launch vehicles. A vehicle is assembled from four interchangeable models (gravity, aerodynamics, propulsion and mass) and propagated by integrators that respect the geometry of rigid-body motion: the vehicle's orientation is stored and advanced as a rotation, never as a loose array of numbers that has to be repaired afterwards. Every model is written once, as a template over its numeric type, so the same source code serves both plain simulation and the exact derivatives that optimisers and state estimators need. The library ships with the U.S. Standard Atmosphere 1976 (NOAA et al., 1976), WGS84 gravity (National Imagery and Mapping Agency, 2000), DAVE-ML aerodynamic tables (Jackson & Hildreth, 2002), multi-stage propulsion, a trim solver, export to FMI co-simulation units, and seventeen executable verification cases from NASA's 6-DOF check-case suite (Murri et al., 2015).

Statement of need

6-DOF simulation underpins vehicle design, control-law verification and trajectory optimisation. Three habits of existing codes make them hard to reuse for that work. First, attitude is usually packed into a flat state vector with a general-purpose ODE solver, and the unit-norm or orthonormality constraint is restored by periodic re-normalisation. That projection lives outside the integrator, so it is absent from any propagation Jacobian, which is a silent modelling error inside a filter or optimiser. Second, codes used for estimation or optimisation tend to keep a second, hand-maintained implementation of the dynamics for derivatives, which drifts from the simulated one; finite differences are the usual fallback. Third, aerodynamic, propulsion and mass models are often hard-wired into the integration loop, so adding a vehicle means editing the solver.

Aetherion is written for researchers in guidance, navigation and control, trajectory optimisation and flight simulation who need a 6-DOF plant that can be trusted numerically and handed to a gradient-based algorithm without being re-implemented. Researchers who do not use C++ can still use an Aetherion vehicle as a plant through its FMI export.

State of the field

Open-source tools cover parts of this space, each for a different purpose. JSBSim (Berndt, 2004) is a mature atmospheric flight dynamics model oriented towards real-time flight simulation, with neither algorithmic differentiation nor launch-vehicle staging as design goals. RBDL (Felis, 2017) and Pinocchio (Carpentier et al., 2019) implement Featherstone's algorithms to a high standard, and Pinocchio offers analytical derivatives, but both target articulated robots and

DRAFT

carry no atmosphere, gravity-field or staging models. manif (Deray & Solà, 2020) provides Lie-group primitives with Jacobians, but is a library of group operations rather than of vehicle dynamics. MATLAB's Aerospace Blockset (The MathWorks, Inc., 2024) is commercial, and NASA's Trick (Penn & Lin, 2016) and POST (Brauer et al., 1977) are simulation frameworks rather than differentiable model libraries, POST being distribution-restricted.

Contributing to one of these was considered and rejected because the properties Aetherion needs are architectural rather than additive. Making JSBSim differentiable would require rewriting its models over a generic scalar type; making a robotics library launch-vehicle capable would add the whole environment and propulsion stack while still leaving its state in a flat parameterisation. Aetherion therefore combines aerospace environment and vehicle models, Lie-group integration and single-source differentiability in one library, and reuses established components (Eigen (Guennebaud et al., 2010), CppAD (Bell, 2025), fmu4cpp (Hatledal, 2025b)) wherever they fit.

Software design

Figure 1 shows the module structure. Three design decisions carry most of the weight, and each is enforced by the type system rather than by convention.

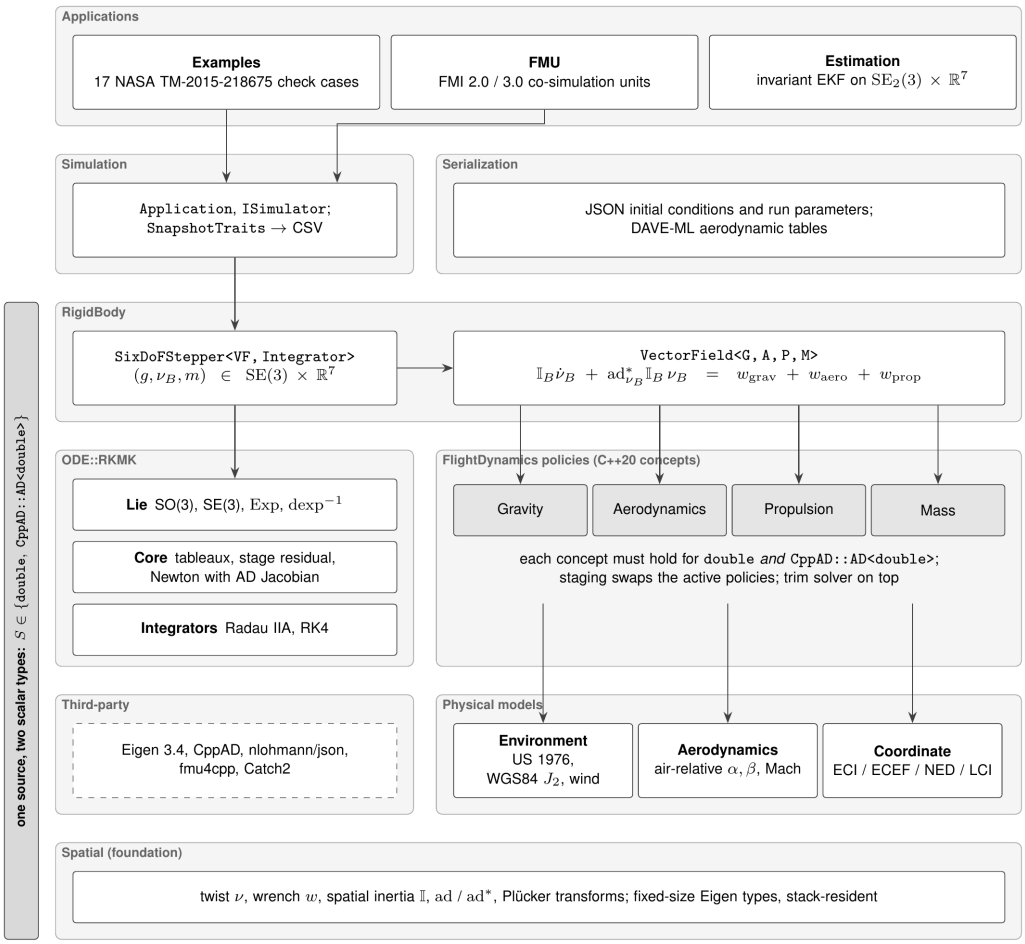


Figure 1: Module structure of Aetherion. Arrows point from a module to the modules it uses. Applications drive a `SixDoFStepper`, which composes a Lie-group integrator with a `VectorField` built from four vehicle policies; everything from the rigid-body layer down is a template over the scalar type S.

One vocabulary. All kinematic and dynamic quantities are spatial 6-vectors (Featherstone, 2008): twists, wrenches, spatial inertia, and the motion and force cross-product operators. The rigid-body equation of motion is a single expression with no separate force and moment bookkeeping, and `Twist`, `Wrench` and `Inertia` are distinct types, so a wrench cannot be passed where a twist is expected.

The state stays on the manifold. The pose is a single element of $\mathrm{SE}(3)$, not a split rotation-plus-position pair, and is advanced by Runge–Kutta–Munthe-Kaas methods (Iserles et al., 2000; Munthe-Kaas, 1998) that perform their stages in the Lie algebra and map back with the exponential. An implicit Radau IIA integrator, solved by Newton iteration with an AD Jacobian, and an explicit RK4 integrator share the same stage machinery. Any type satisfying a `LieGroup` concept can replace $\mathrm{SE}(3)$ without touching it, and the estimation module reuses this to run an invariant extended Kalman filter on $\mathrm{SE}_2(3)$.

One source, two scalar types. Every model, from the atmosphere to the exponential map, is a template over a scalar type S. With $S =$ `double` it simulates; with $S =$ `CppAD::AD<double>` it produces exact derivatives through the same code. The four vehicle policies are C++20 concepts that must hold for *both* types, so a model that would tape incorrectly, for instance by branching on a floating-point comparison, fails to compile at its definition rather than yielding a wrong Jacobian at run time. The cost is a discipline visible throughout the code: environment models are written branch-free or with AD-aware conditional expressions.

A vehicle is a type, `VectorField<Gravity, Aero, Propulsion, Mass>`, and the stepper is `SixDoFStepper<VectorField, Integrator>`; changing the vehicle or the integrator changes template arguments, not solver code. Stage systems and Jacobians are fixed-size Eigen objects, so the integration hot path does not allocate apart from the vector round-trips required by the CppAD tape interface, keeping real-time and hardware-in-the-loop use open. Output columns are selected at compile time, including a layout identical to the NASA reference files so that verification needs no column mapping.

Research impact statement

Aetherion implements seventeen check cases from NASA TM-2015-218675 (Murri et al., 2015), from analytic point masses through closed-loop F-16 manoeuvres to a two-stage ascent to 232 km. They are built and run in continuous integration and, in every case, Aetherion lies within the envelope spanned by NASA's own independent reference implementations. The convergence-order and constraint-preservation studies are regression tests that emit the data they are judged by, so a change that degrades the integrator fails the build. Each pull request is also built on Linux and Windows and run through roughly 750 Catch2 test cases, address and undefined-behaviour sanitizers, Clang-Tidy and a complexity budget.

The library is used in the author's group for launch-vehicle ascent studies and control-law verification. Vehicles are exported as FMI 2.0 and 3.0 units, smoke-tested against the Ecos co-simulation engine (Hatledal, 2025a) and imported into Simulink in CI, which lets groups use Aetherion inside existing toolchains without adopting its C++ API.

AI usage disclosure

Claude (Anthropic) was used to assist with software development and with editing this manuscript. All AI-assisted code went through the same pull request process as other changes, including the full test suite, sanitizers, static analysis and the NASA verification cases; the author reviewed all generated text and code and takes full responsibility for it.

DRAFT

The Journal of Open Source Software

Acknowledgements

The author thanks the NASA Engineering and Safety Center for making the TM-2015-218675 reference data publicly available.

References

Bell, B. M. (2025). *CppAD: A package for differentiation of C++ algorithms*. https://github.com/coin-or/CppAD

Berndt, J. S. (2004). JSBSim: An open source flight dynamics model in C++. *AIAA Modeling and Simulation Technologies Conference and Exhibit*. https://doi.org/10.2514/6.2004-4923

Brauer, G. L., Cornick, D. E., & Stevenson, R. (1977). *Capabilities and applications of the program to optimize simulated trajectories (POST)* (NASA CR-2770). NASA.

Carpentier, J., Saurel, G., Buondonno, G., Mirabel, J., Lamiraux, F., Stasse, O., & Mansard, N. (2019). The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. *IEEE/SICE International Symposium on System Integration (SII)*, 614–619. https://doi.org/10.1109/SII.2019.8700380

Deray, J., & Solà, J. (2020). Manif: A micro Lie theory library for state estimation in robotics applications. *Journal of Open Source Software*, *5*(46), 1371. https://doi.org/10.21105/joss.01371

Featherstone, R. (2008). *Rigid body dynamics algorithms*. Springer. https://doi.org/10.1007/978-1-4899-7560-7

Felis, M. L. (2017). RBDL: An efficient rigid-body dynamics library using recursive algorithms. *Autonomous Robots*, *41*(2), 495–511. https://doi.org/10.1007/s10514-016-9574-0

Guennebaud, G., Jacob, B., & others. (2010). *Eigen v3*. https://eigen.tuxfamily.org

Hatledal, L. I. (2025a). Ecos: An accessible and intuitive co-simulation framework. *Journal of Open Source Software*, *10*(110), 8182. https://doi.org/10.21105/joss.08182

Hatledal, L. I. (2025b). *fmu4cpp: Export C++ models as FMI 2.0/3.0 functional mock-up units*. https://github.com/markaren/fmu4cpp

Iserles, A., Munthe-Kaas, H. Z., Nørsett, S. P., & Zanna, A. (2000). Lie-group methods. *Acta Numerica*, *9*, 215–365. https://doi.org/10.1017/S0962492900002154

Jackson, E. B., & Hildreth, B. L. (2002). Flight dynamic model exchange using XML. *AIAA Modeling and Simulation Technologies Conference and Exhibit*. https://doi.org/10.2514/6.2002-4482

Munthe-Kaas, H. (1998). Runge–Kutta methods on Lie groups. *BIT Numerical Mathematics*, *38*(1), 92–111. https://doi.org/10.1023/A:1021919121541

Murri, D. G., Jackson, E. B., & Shelton, R. O. (2015). *Check-cases for verification of 6-degree-of-freedom flight vehicle simulations* (NASA Technical Memorandum NASA/TM-2015-218675). NASA Langley Research Center. https://ntrs.nasa.gov/citations/20150001263

National Imagery and Mapping Agency. (2000). *Department of defense world geodetic system 1984: Its definition and relationships with local geodetic systems* (NIMA TR8350.2, 3rd ed.). National Imagery; Mapping Agency.

NOAA, NASA, & U.S. Air Force. (1976). *U.S. Standard atmosphere, 1976* (NOAA-S/T 76-1562). U.S. Government Printing Office.

Penn, J. M., & Lin, A. S. (2016). The Trick simulation toolkit: A NASA/open source framework for running time based physics models. *AIAA Modeling and Simulation Technologies*

DRAFT

Conference. https://doi.org/10.2514/6.2016-1187

The MathWorks, Inc. (2024). *Aerospace blockset user's guide*. The MathWorks, Inc. https://www.mathworks.com/products/aerospace-blockset.html

DRAFT